

投资的收益和风险：含最低交易费的组合优化

摘要

针对含最低交易费的资产配置问题，建立银行存款与风险资产之间的收益—风险组合优化模型。将资金总额和可承受风险作为决策参数，在给定风险上限下最大化净收益，构造供公司选择的投资方案。

将购买金额、交易费和银行存款共同纳入预算；不购买时费用为零，正购买时费用为费率乘购买金额与门槛金额的较大值。利用二元启用变量建立混合整数线性规划，并以最大单项风险损失额占初始资金比例作为风险指标。

当可用资金为 100 万元时，四资产风险上限 1% 时，购买 S1 400,000.00 元、S2 584,313.73 元，净收益 219,019.61 元，净收益率 21.902%；十五资产风险上限 10% 时，净收益率为 33.534%。两组各计算 61 个风险上限；代表方案均与独立解析上界吻合。

四资产进一步通过 81 种费用分段的枚举与连续背包计算独立核验。资金和风险参数情景揭示：最低交易费在小资金下会改变组合；将风险理解为所选资产最大风险率时，方案会明显改变。因此结果仅适用于所采用定义与题目参数，模型同时给出最低交易费生效条件，为不同资金规模下的组合选择提供依据。

关键词：投资组合；混合整数规划；最低交易费；风险约束；独立验证

1 问题分析与数据核对

原题的两个要求分别是：对四种资产设计收益与风险的组合方案；讨论一般情形并对十五种资产计算。原始数据来自全国组委会官方历史赛题汇编第 11 页^[1]。收益率、风险损失率和费率在计算时由百分数转为小数；门槛金额以元计。

资金总额 M 与公司风险偏好未提供。本文取 $M=1,000,000$ 元作为数值分析基准，并考察 $M=100$ 、 $1,000$ 、 $10,000$ 元。资金总额保留为模型参数，各风险上限对应一个收益最优方案；公司可根据可承受风险选择配置。

表 1 四资产题设参数

资产	收益率 %	风险率 %	费率 %	门槛 元
S1	28	2.5	1	103
S2	21	1.5	2	198
S3	23	5.5	4.5	52
S4	25	2.6	6.5	40

2 假设、定义与符号

采用单时期、无卖空、资金可连续分割；题目收益率与风险率视为本期给定参数。银行利率 $r_0=5\%$ ，无风险和交易费。没有观测时间序列，不估计相关性、收益分布或市场预测精度，也不将风险率解释成违约概率。税收、流动性与跨期再平衡不在题目数据内。

表 2 符号与单位

符号	含义与单位
M	初始可用资金，元
x_0/x_i	银行存款 / 第 i 项购买本金，元
$r_i/q_i/p_i$	收益率、题设风险损失率、交易费率，无量纲
u_i/f_i	费用计费门槛 / 实际交易费，元
y_i	启用变量，0 或 1
ρ/R	风险上限 / 最大单项风险暴露额占 M 的比例

风险采用最大单项金额暴露占初始资金的比例，而非所选资产风险率的最大值。这是本文为体现分散投资效果所采用的操作定义，不是由题目数据识别出的

损失分布。原文的另一种解释在第 7 节比较。 u_1 是最低计费基数，不是最低购买额；交易费函数为：

$$f_i(0) = 0, \quad f_i(x_i) = p_i \max(x_i, u_i) \quad (x_i > 0) \quad (1)$$

单时期、连续分割和忽略税收使预算模型可线性化；若交易要求整手购买，则需增加整数购买单位，本文结果不再直接适用。若费用另有资金支付或原题收益率已扣费，也须改变下面的现金流模型。

3 一般模型与精确求解

3.1 现金流与风险约束

交易费在购买时从初始资金支付。期末财富由银行本息及风险资产本息组成；期末财富减去初始投入，得到净利润。因此费用只计入利润一次，不能因为预算也含费用就再重复扣减。

$$W = (1 + r_0)x_0 + \sum_{i=1}^n (1 + r_i)x_i, \quad P = W - M \quad (2)$$

$$P = r_0x_0 + \sum_{i=1}^n r_ix_i - \sum_{i=1}^n f_i \quad (3)$$

$$x_0 + \sum_{i=1}^n (x_i + f_i) = M, \quad x_0, x_i \geq 0 \quad (4)$$

$$R(\mathbf{x}) = \max_{1 \leq i \leq n} \frac{q_i x_i}{M} \leq \rho \quad (5)$$

收益率为净利润除以初始资金。风险指标仅约束资金暴露；没有收益与损失的联合分布，故不把风险参数额外扣入期望收益。给定不同风险上限得到一族条件最优方案，全银行提供净收益率 5% 的可行基线。

3.2 混合整数线性表示

将金额除以初始资金归一化，避免资金尺度使约束系数悬殊。对每种资产加入二元启用变量；其具体定义和优化问题如下（所有逐项约束均对 $i=1, \dots, n$ 成立）：

$$v_i = \frac{x_i}{M}, \quad b = \frac{x_0}{M}, \quad h_i = \frac{f_i}{M}, \quad t_i = \frac{u_i}{M} \quad (6)$$

$$\max \quad r_0 b + \sum_{i=1}^n (r_i v_i - h_i) \quad (7)$$

$$b + \sum_{i=1}^n (v_i + h_i) = 1, \quad b, v_i, h_i \geq 0 \quad (8)$$

$$0 \leq v_i \leq y_i, \quad h_i \geq p_i v_i, \quad h_i \geq p_i t_i y_i \quad (9)$$

$$h_i \leq p_i \max(1, t_i) y_i, \quad q_i v_i \leq \rho, \quad y_i \in \{0, 1\} \quad (10)$$

当 $y_i=0$ 时购买额与费用均为零；当 $y_i=1$ 时费用至少等于两种计费值的较大者。由于过高费用既直接减少利润又占用可生息预算，最优解不会多付费用，故在最优解处该表示与分段收费等价。即使启用变量为 1 而购买额为零，取消启用并把费用转存银行也会严格增加收益，故这种状态不会成为最优解。费用上界利用归一化购买额不超过 1 推导，避免任意超大的常数。

用 SciPy 的 milp 接口^[2]和 HiGHS 求解^[3]，相对最优间隙设为 $1e-9$ ，单次上限 20 秒；本次均求得最优状态。每组扫描 61 个风险上限，四资产范围 0 到 3%，十五资产范围 0 到 60%。离散扫描展示收益包络，不声称给出全部连续前沿或所有并列最优解。

4 四资产结果

表 3 四资产代表方案收益

风险上限	实际风险	净收益率	净收益 元	银行 元
0.500%	0.500%	17.764%	177,638.05	158,192.31
1.000%	1.000%	21.902%	219,019.61	0.00
2.000%	2.000%	25.176%	251,764.71	0.00

表 4 四资产购买本金及交易费用（元）

风险上限	S1 本金	S2 本金	S3 本金	S4 本金	费用合计
0.500%	200,000.00	333,333.33	90,909.09	192,307.69	25,257.58
1.000%	400,000.00	584,313.73	0.00	0.00	15,686.27
2.000%	800,000.00	188,235.29	0.00	0.00	11,764.71

风险上限 0.5% 时四种资产都有配置，并保留存款；到 1% 时方案集中在 S1 与 S2。该结果说明“分散越多越好”不是收益最优的通用规则；具体组合取决于风险限制和净收益机会成本。不同风险上限不能直接作优劣排名。

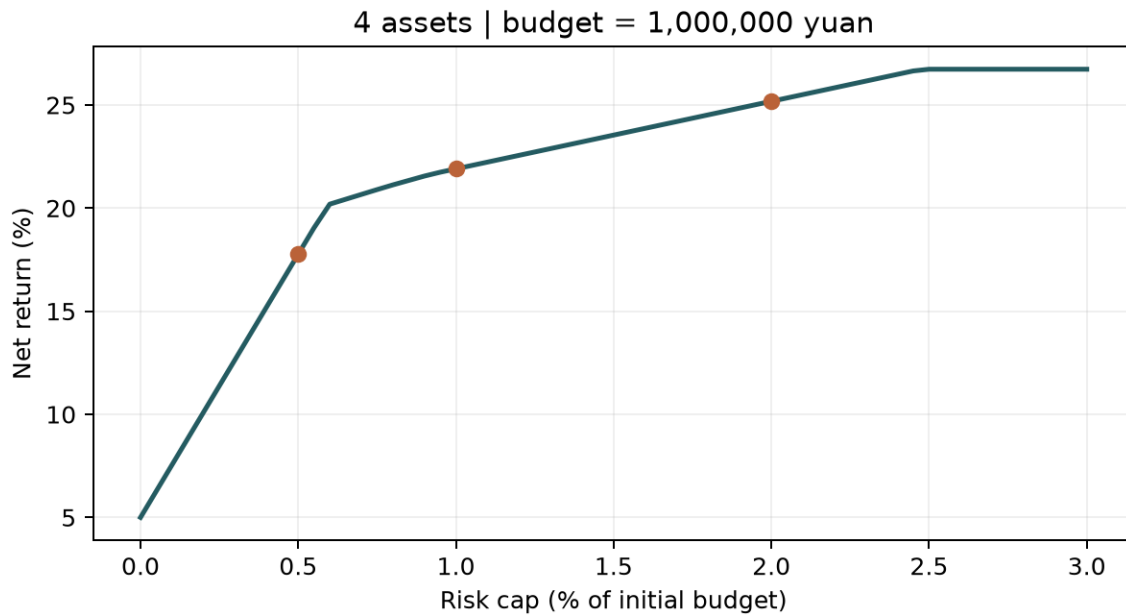


图 1 四资产风险上限与最优净收益率；点为代表方案，横轴为约束上限。

5 十五资产结果与一般扩展

表 5 十五资产代表方案收益

风险上限	实际风险	净收益率	净收益 元
5.000%	5.000%	26.433%	264,333.07
10.000%	10.000%	33.534%	335,341.21
20.000%	20.000%	35.862%	358,621.68

十五资产采用与四资产完全相同的模型，全部参数见支撑材料 data/assets15.csv。一般 n 情形有 $3n+1$ 个变量，其中 n 个二元变量；新增资产不求更换方法。规模更大时求解耗时与启用组合有关，应保留实际最优间隙，不能只凭求解器返回可行解宣称全局最优。

表 6 十五资产代表配置

风险上限 10% 的启用资产	购买本金 元	交易费 元
S3	166,666.67	10,000.00
S7	147,058.82	8,235.29
S9	182,598.19	4,930.15
S10	250,000.00	7,250.00
S13	217,391.30	5,869.57

该方案交易费合计 36,285.01 元。无风险限制时，四资产最优净收益率 26.733%，十五资产为 40.943%；相应集中投资的风险更高。没有公司偏好输入，本文不指定一个“最平衡”方案。

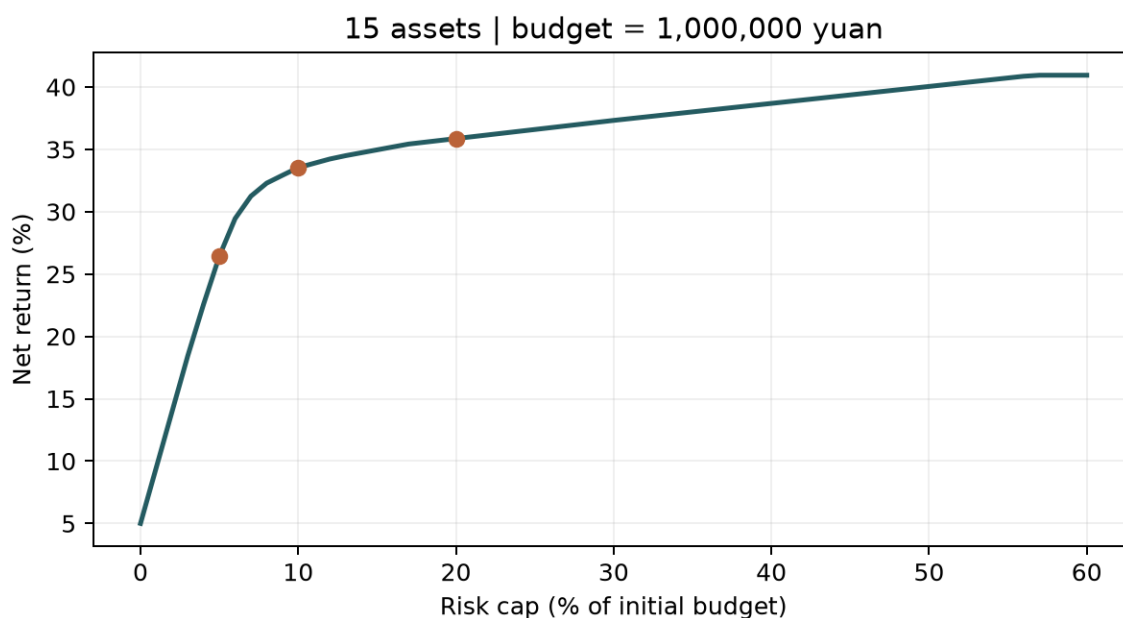


图 2 十五资产最优收益包络；平台区不表示风险上限全部被用尽。

6 独立核验与最优性证据

首先从原始 CSV 独立重算预算、分段费用、利润与实际风险，检查非负性和约束；随后检查零风险退化为全银行、收益包络单调以及无风险限制下的解析答案。

四资产的核验不调用优化求解器：逐项取“不买 / 低于门槛 / 高于门槛”三种状态，共 $3^4=81$ 种。在每个状态中，费用已是仿射函数，资金约束成为带上下限的连续背包；先分配下限，再按边际利润与边际预算比从高到低分配，获得该状态

的确切最优值。取所有状态的最大值，与 MILP 比较。

6.1 比例费用松弛与解析上界

把最低计费门槛去掉，仅保留比例费用，则原来的任意投资组合在松弛问题中花费不更多，可把节省的费用存入银行，净收益不降低。因此松弛最优值是原问题的上界，不是另一个经验近似答案。将银行余额由预算消去，得到：

$$w_i = 1 + p_i, \quad g_i = r_i - r_0 - (1 + r_0)p_i \quad (11)$$

$$\max \quad r_0 + \sum_{i=1}^n g_i v_i \quad (12)$$

$$\sum_{i=1}^n w_i v_i \leq 1, \quad 0 \leq v_i \leq \frac{\rho}{q_i} \quad (13)$$

这里每增加一单位购买本金，预算消耗为 w_i ，扣除银行机会成本后的增量净收益为 g_i 。只考虑正增益资产，按单位预算的增益从高到低分配；每分配一种资产后更新剩余预算 B ：

$$\eta_i = \frac{g_i}{w_i}, \quad v_i = \min\left(\frac{B}{w_i}, \frac{\rho}{q_i}\right) \quad (14)$$

交换论证：若较高增益比资产尚未到风险上限，而较低增益比资产获得正的预算，转移同量预算至前者将提高目标值，故最优分配必须按此排序。依次填满单项上限或耗尽预算，即得到松弛问题的精确解 U 。若净增益非正，保留银行存款即可。

$$\frac{P_{\text{feasible}}}{M} \leq \frac{P^*}{M} \leq U(\rho) \quad (15)$$

当原问题可行解的收益等于上界 U 时，两边夹逼证明其全局最优。本次两组各三个代表方案均达到上界；小资金方案可能落在最低费用段，不能自动使用上界相等的证书。

6.2 一般资金规模下的解析条件

比例费用松弛的最优配置记为归一化购买额 v^* 。对所有启用资产，若购买本金不低于最低计费门槛，则原问题与松弛问题的费用一致。给定风险上限，定义充分资金阈值：

$$M_{\text{crit}} = \max_{i: v_i^* > 0} \frac{u_i}{v_i^*}, \quad M \geq M_{\text{crit}} \quad (16)$$

当资金不低于该阈值时，比例费用解在原模型中可行并达到上界，因而也是原问题最优解，净收益率与资金总额无关；若没有资产启用，则阈值记为零。低于阈值仅说明该证书失效，仍应使用分段费用模型求解，不能直接断言收益必定下降。

四资产风险上限 1% 时，资金阈值约为 338.86 元；十五资产风险上限 10% 时约为 2601.34 元。为避免金额四舍五入低于边界，可分别取 339 元与 2,602 元作为充分资金条件。该结论解释了第 7 节中资金超过门槛后最优收益率保持不变的现象。

6.3 数值证据与检验边界

本次 12/12 项检查通过。支撑材料 `reference/checks.json` 记录实际误差、约束残差及证书间隙；这 12 项包含可行性、会计恒等式和性质检验，并非 12 种独立求解算法；真正独立的最优性核验为费用分段枚举与比例费用解析上界。数值误差以 $1e-7$ 的无量纲容差处理，金额表按分四舍五入，极小负银行残差在展示时记为零。

7 资金规模、参数与解释敏感性

四资产，风险上限 1%

表 7 四资产资金规模情景

资金 元	净收益率	交易费 元
100.00	18.426%	5.97
1,000.00	21.902%	15.69
10,000.00	21.902%	156.86
1,000,000.00	21.902%	15,686.27

十五资产，风险上限 10%

表 8 十五资产资金规模情景

资金 元	净收益率	交易费 元
100.00	6.082%	14.01
1,000.00	30.388%	56.93
10,000.00	33.534%	362.85
1,000,000.00	33.534%	36,285.01

资金情景是自行设定的压力检查，100 元情景偏离原题“相当大资金”的描述，专用于检验门槛。资金大到所有启用投资超过门槛后，比例归一化使最优收益率不再随 M 改变；资金小时固定最低计费会影响收益和选择。

四资产：将全部 q_1 乘以 0.9、1、1.1，在同一风险上限下重新求解，收益率依次为 22.266%、21.902%、21.591%。

十五资产：将全部 q_1 乘以 0.9、1、1.1，在同一风险上限下重新求解，收益率依次为 33.917%、33.534%、32.977%。

这三个乘数没有经验分布依据，只是参数情景，不是置信区间或真实发生概率。风险率增大使可行域缩小，最优收益不应增大，本次检查符合该性质。

若将总体风险定义为已选资产的最大 q_1 ，则四资产 1% 上限下无风险资产合格；十五资产 10% 上限下虽有低风险资产，但其费用后的收益不优于银行，因此两者均选择全银行并获得 5%。解释改变会从根本上改变结果，因此风险指标的操作定义应与公司的决策目标一致。

8 模型评价与条件建议

优点是费用与预算一致、两组问题共用一般模型、有可复现输出与独立最优性证据。限制是最大单项损失只是一种题设风险代理，不是组合方差、尾部总损失或概率保证；缺少相关性与收益分布，不能代表真实市场投资。返回的是条件最优方案之一，而非所有可能的并列方案。

建议公司先确认可用资金、可承受的损失额与风险定义，再选对应风险上限的组合；若真实目标改成总损失、流动性或多时期决策，需要重新建模。提高模型复杂度不能弥补缺失数据。

AI工具使用声明

本论文使用了 AI 工具，主要用于模型推导、代码编写、计算验证、结果分析及论文编排，详细使用情况见支撑材料 AI工具使用详情.pdf。

参考文献

- [1] 全国大学生数学建模竞赛组委会. 1992—2000 年全国大学生数学建模竞赛赛题汇编：1998 年 A 题，PDF 第 11 页[EB/OL]. [2026-10-07]. https://www.mcm.edu.cn/upload_cn/node/1/SkAh1A7Q6f2dd01e58aa621f920e79f45cd5d255.pdf
- [2] SCIPY DEVELOPERS. scipy.optimize.milp: SciPy API reference[EB/OL]. [2026-10-07]. <https://docs.scipy.org/doc/scipy/reference/generated/scipy.optimize.milp.html>
- [3] HiGHS DEVELOPMENT TEAM. HiGHS: high performance software for linear optimization[EB/OL]. [2026-10-07]. <https://highs.dev/>

附录 A 数据与支撑材料文件列表

十五资产参数来自原题第二张表^[1]，供不运行程序的读者复核；收益、风险与费用列单位均为百分数，门槛为元。

表 9 十五资产题设参数

资产	收益率 %	风险率 %	费率 %	门槛 元
S1	9.6	42	2.1	181
S2	18.5	54	3.2	407
S3	49.4	60	6	428
S4	23.9	42	1.5	549
S5	8.1	1.2	7.6	270
S6	14	39	3.4	397
S7	40.7	68	5.6	178
S8	31.2	33.4	3.1	220
S9	33.6	53.3	2.7	475
S10	36.8	40	2.9	248
S11	11.8	31	5.1	195
S12	9	5.5	5.7	320
S13	35	46	2.7	267
S14	9.4	5.3	4.5	328
S15	15	23	7.6	131

supporting_materials.zip 含以下文件；reproduced/ 为重跑时新生成，不纳入原提交包。各文件对应本文的数据、数值结果与计算过程。

README.md	build_submission.py
code/model.py	code/validate.py
data/assets15.csv	data/assets4.csv
equations.json	figures/fifteen-frontier.png
figures/four-frontier.png	make_equations.py
reference/checks.json	reference/fifteen-frontier.csv
reference/four-frontier.csv	reference/results.json
report-requirements.txt	requirements.txt
run_demo.py	runtime-versions.json
sources.json	AI工具使用详情.pdf

附录 B 完整源程序

以下列出建模、独立核验及结果复现的完整源码。程序按支撑包运行说明执行；公式生成与论文编排脚本另附于支撑包。

code/model.py

```
"""Original solution to a historical CUMCM problem; parameters are demo scenarios."""
import argparse
import csv
import json
from pathlib import Path
import numpy as np
from scipy.optimize import milp, Bounds, LinearConstraint
import matplotlib
matplotlib.use('Agg')
import matplotlib.pyplot as plt

def load(name):
    rows=list(csv.DictReader(open(Path('raw/data')/name,encoding='utf-8')))
    return rows, np.array([[float(d[k]) for k in ['return_pct','risk_pct','fee_pct','thresh
old_yuan']] for d in rows])

def solve(data, budget, cap, risk_scale=1.0, interpretation='amount'):
    n=len(data);r,q,p,u=data.T.copy();r/=100;q=q/100*risk_scale;p/=100;u/=budget
    # v = [bank, investments, fees, activation binaries], all money scaled by budget.
    size=1+3*n;ix=np.arange(1,1+n);jf=ix+n;ky=jf+n
    c=np.r_[-.05,-r,np.ones(n),np.zeros(n)]
    upper=np.r_[1,np.ones(n),p*np.maximum(1,u),np.ones(n)]
    if interpretation=='rate':
        upper[ix[q>cap+1e-12]]=0
    rows=[];lb=[];ub=[]
    a=np.zeros(size);a[0]=1;a[ix]=1;a[jf]=1
    rows.append(a);lb.append(1);ub.append(1)
    for i in range(n):
        for coeffs,hi in [(int(ix[i]):p[i],int(jf[i]):-1),0),
                          ((int(ky[i]):p[i]*u[i],int(jf[i]):-1),0),
                          ((int(ix[i]):1,int(ky[i]):-1),0),
                          ((int(jf[i]):1,int(ky[i]):-upper[jf[i]]),0)]:
            a=np.zeros(size)
            for j,value in coeffs.items():a[j]=value
            rows.append(a);lb.append(-np.inf);ub.append(hi)
    if interpretation=='amount':
        a=np.zeros(size);a[ix[i]]=q[i]
        rows.append(a);lb.append(-np.inf);ub.append(cap)
    res=milp(c,integrality=np.r_[np.zeros(1+2*n),np.ones(n)],
            bounds=Bounds(np.zeros(size),upper),
            constraints=LinearConstraint(np.array(rows),np.array(lb),np.array(ub)),
            options={'mip_rel_gap':1e-9,'time_limit':20})
    if not res.success:raise RuntimeError(res.message)
    x=np.maximum(res.x[ix],0)*budget
    fees=np.where(x>1e-7,p*np.maximum(x,data[:,3]),0)
    bank=budget-x.sum()-fees.sum()
    profit=.05*bank+np.dot(r,x)-fees.sum()
    risk=float(np.max(q*x)/budget) if interpretation=='amount' else float(max(q[x>1e-7],def
ault=0))
    return {'budget_yuan':budget,'risk_cap':float(cap),'risk_scale':risk_scale,
```

```

        'interpretation':interpretation,'investments_yuan':x.tolist(),
        'fees_yuan':fees.tolist(),'bank_yuan':float(bank),
        'profit_yuan':float(profit),'net_return':float(profit/budget),
        'actual_risk':risk,'mip_gap':float(res.mip_gap),
        'solver_message':res.message}

def main():
    parser=argparse.ArgumentParser();parser.add_argument('--output',type=Path,required=True
)
    out=parser.parse_args().output
    results={'historical_demo':True,'bank_rate':.05,'primary_budget_yuan':1e6,
        'risk_definition':'max_i(q_i*x_i)/M; x_i is purchased principal; fees paid fro
m M',
        'groups':{},'sensitivity':{},'alternative_interpretation':{}}
    for key,name,end,caps in [('four','assets4.csv',.03,[.005,.01,.02]),
        ('fifteen','assets15.csv',.6,[.05,.1,.2])]:
        rows,data=load(name)
        frontier=[solve(data,1e6,cap) for cap in np.linspace(0,end,61)]
        selected=[solve(data,1e6,cap) for cap in caps]
        results['groups'][key]={ 'assets':rows,'frontier':frontier,'selected':selected,
            'unrestricted':solve(data,1e6,1.0)}
        results['sensitivity'][key]={ 'budget':[solve(data,m,caps[1]) for m in [100,1000,100
00,1e6]],
            'risk_stress':[solve(data,1e6,caps[1],scale) for scale in [.9,1,1.1]]}
        results['alternative_interpretation'][key]=solve(data,1e6,caps[1],interpretation='r
ate')
        with open(out/f'{key}-frontier.csv','w',newline='') as f:
            w=csv.writer(f);w.writerow(['risk_cap','actual_risk','net_return','profit_yuan'
,'bank_yuan'])
            w.writerows([d[k] for k in ['risk_cap','actual_risk','net_return','profit_yuan'
,'bank_yuan']] for d in frontier)
            fig,ax=plt.subplots(figsize=(6.8,3.8))
            ax.plot([100*d['risk_cap'] for d in frontier],[100*d['net_return'] for d in frontie
r],color='#245B61',lw=2)
            ax.scatter([100*d['actual_risk'] for d in selected],[100*d['net_return'] for d in s
elected],color='#BA6238',zorder=3)
            ax.set(xlabel='Risk cap (% of initial budget)',ylabel='Net return (%)',title=f'{len
(rows)} assets | budget = 1,000,000 yuan')
            ax.grid(alpha=.2);fig.tight_layout();fig.savefig(out/f'{key}-frontier.png',dpi=180)
;plt.close(fig)
        (out/'results.json').write_text(json.dumps(results,ensure_ascii=False,indent=2,allow_na
n=False))

if __name__=='__main__':main()

```

code/validate.py

```
"""Independent arithmetic, fee-regime enumeration and analytical upper bounds."""
import argparse
import csv
import itertools
import json
from pathlib import Path
import numpy as np

def load(name):
    with open(Path('raw/data')/name, encoding='utf-8') as f:
        rows=list(csv.DictReader(f))
        return np.array([[float(d[k]) for k in ['return_pct', 'risk_pct', 'fee_pct', 'threshold_yuan']] for d in rows])

def regime_optimum(data, budget, cap):
    # off / minimum-fee region / proportional-fee region: solve each box knapsack by ratios
    .
    r,q,p,u=data.T.copy();r/=100;q/=100;p/=100;u/=budget
    best=.05
    for regimes in itertools.product(range(3), repeat=len(data)):
        lo=[];hi=[];weight=[];gain=[];fixed=0.
        for i,state in enumerate(regimes):
            risk_upper=cap/q[i]
            if state==0:a=b=0.;w=1.;g=0.
            elif state==1:
                a=0.;b=min(u[i], risk_upper);w=1.;g=r[i]-.05
                fixed+=p[i]*u[i]
            else:
                a=u[i];b=risk_upper;w=1+p[i];g=r[i]-.05-1.05*p[i]
                lo.append(a);hi.append(b);weight.append(w);gain.append(g)
        if any(b<a-1e-12 for a,b in zip(lo,hi)):continue
        remaining=1-fixed-sum(w*a for w,a in zip(weight,lo))
        if remaining < -1e-12:continue
        value=.05-1.05*fixed+sum(g*a for g,a in zip(gain,lo))
        for i in sorted(range(len(data)),key=lambda j:gain[j]/weight[j],reverse=True):
            if gain[i]<=0:continue
            extra=min(max(remaining,0)/weight[i],hi[i]-lo[i])
            value+=gain[i]*extra;remaining-=weight[i]*extra
        best=max(best,value)
    return best

def relaxed_upper(data, cap, scale=1.):
    # Ignore minimum fees. Nonnegative bank is a budget slack, so fractional knapsack is exact.
    r,q,p,_=data.T.copy();r/=100;q=q/100*scale;p/=100
    w=1+p;g=r-.05-1.05*p;left=1.;value=.05
    for i in sorted(range(len(data)),key=lambda j:g[j]/w[j],reverse=True):
        if g[i]<=0:continue
        xi=min(left/w[i], cap/q[i]);value+=g[i]*xi;left-=w[i]*xi
    return value

def main():
    a=argparse.ArgumentParser();a.add_argument('--results',type=Path,required=True);a.add_argument('--output',type=Path,required=True)
    args=a.parse_args();result=json.loads(args.results.read_text());checks=[]
```

```

def check(name,passed,evidence):checks.append({'name':name,'passed':bool(passed),'evidence':evidence))
for key,file in [('four','assets4.csv'),('fifteen','assets15.csv')]:
    data=load(file);group=result['groups'][key]
    samples=group['frontier']+group['selected']+group['unrestricted']+result['sensitivity'][key]['budget']+result['sensitivity'][key]['risk_stress']+result['alternative_interpretation'][key]
    residuals=[];errors=[];violation=[]
    for d in samples:
        M=d['budget_yuan'];x=np.array(d['investments_yuan']);r,q,p,u=data.T.copy();r/=100;q=q/100*d['risk_scale'];p/=100
        fee=np.where(x>1e-7,p*np.maximum(x,u),0)
        profit=.05*d['bank_yuan']+np.dot(r,x)-fee.sum()
        risk=max(q[x>1e-7],default=0) if d['interpretation']=='rate' else max(q*x)/M
        residuals.append(abs(d['bank_yuan']+x.sum()+fee.sum()-M)/M)
        errors.extend([abs(profit-d['profit_yuan'])/M,np.max(abs(fee-d['fees_yuan']))/M,abs(risk-d['actual_risk'])])
        violation.append(max(0,-min(x)/M,-d['bank_yuan']/M,risk-d['risk_cap']))
    check(key+' feasibility and accounting',max(residuals+errors+violation)<1e-7,
        f'{len(samples)} portfolios; budget residual={max(residuals):.3g}, accounting error={max(errors):.3g}, violation={max(violation):.3g}')
    front=group['frontier'];check(key+' frontier and bank boundary',
        abs(front[0]['net_return']-.05)<1e-9 and all(b['net_return']>=a['net_return']-1e-8 for a,b in zip(front,front[1:])),
        f'zero risk gives {front[0]["net_return"]:.9f}; {len(front)} caps checked for nondecreasing optimal return')
    unrestricted=max([.05]+[(ri/100-pi/100)/(1+pi/100) for ri,_,pi,_ in data])
    check(key+' unrestricted analytical answer',abs(unrestricted-group['unrestricted']['net_return'])<1e-8,
        f'best single-asset proportional return={unrestricted:.12f}; fees above thresholds, computed={group["unrestricted"]["net_return"]:.12f}')
    selected=group['selected'];gaps=[relaxed_upper(data,d['risk_cap'])-d['net_return']
    for d in selected]
    check(key+' analytical optimality certificate',max(abs(g) for g in gaps)<1e-7,
        f'Ignoring minimum fees gives a valid upper bound; selected-point gaps={gaps}')
    if key=='four':
        exact=front+selected+[group['unrestricted']]
        result['sensitivity'][key]['budget']
        gaps=[abs(regime_optimum(data,d['budget_yuan'],d['risk_cap'])-d['net_return'])
    for d in exact]
    check('four exhaustive fee-regime optimum',max(gaps)<1e-7,
        f'3^4=81 fee regimes per portfolio; {len(exact)} cases; max return gap={max(gaps):.3g}; no optimization solver used')
    stress=result['sensitivity'][key]['risk_stress']
    check(key+' risk stress direction',all(b['net_return']<=a['net_return']+1e-8 for a,b in zip(stress,stress[1:])),
        'risk multiplier 0.9/1.0/1.1 returns='+str([d['net_return'] for d in stress]))
    # The supplied asset table has no stochastic observations; test a fee discontinuity explicitly.
    p=.02;u=198
    check('fee discontinuity and threshold semantics',p*max(.001,u)==3.96 and p*max(198,u)=3.96,
        'S2: investment=0 has fee=0; investment=0.001 has fee=3.96; investment=198 has fee=3.96. u is not a minimum purchase quantity.')
    args.output.write_text(json.dumps(checks,ensure_ascii=False,indent=2))
    print(json.dumps(checks,ensure_ascii=False,indent=2))
    raise SystemExit(0 if all(d['passed'] for d in checks) else 1)

if __name__=='__main__':main()

```

run_demo.py

```
"""Run the anonymous support package from a clean work directory."""
import json
from pathlib import Path
import shutil
import subprocess
import sys
import tempfile

root=Path(__file__).resolve().parent
with tempfile.TemporaryDirectory(prefix='model-demo-') as temp:
    work=Path(temp);(work/'raw/data').mkdir(parents=True)
    for path in (root/'data').glob('*.csv'):
        shutil.copyfile(path,work/'raw/data'/path.name)
    out=work/'output';out.mkdir()
    subprocess.run([sys.executable,str(root/'code/model.py'),'--output',str(out)],cwd=work,
check=True)
    subprocess.run([sys.executable,str(root/'code/validate.py'),'--results',str(out/'results.json'),
'--output',str(work/'checks.json')],cwd=work,check=True)
    actual=json.loads((out/'results.json').read_text())
    expected=json.loads((root/'reference/results.json').read_text())
    for group in ['four','fifteen']:
        for a,b in zip(actual['groups'][group]['selected'],expected['groups'][group]['selected']):
            assert abs(a['net_return']-b['net_return'])<1e-7
    target=root/'reproduced'
    if target.exists():raise FileExistsError('Refusing to replace reproduced/')
    shutil.copytree(out,target);shutil.copyfile(work/'checks.json',target/'checks.json')
    print('Independent checks passed; selected report results reproduced within 1e-7.')
```